Class 7: Machine Learning 1

Erin Jang (PID: A17713992)

Table of contents

Background

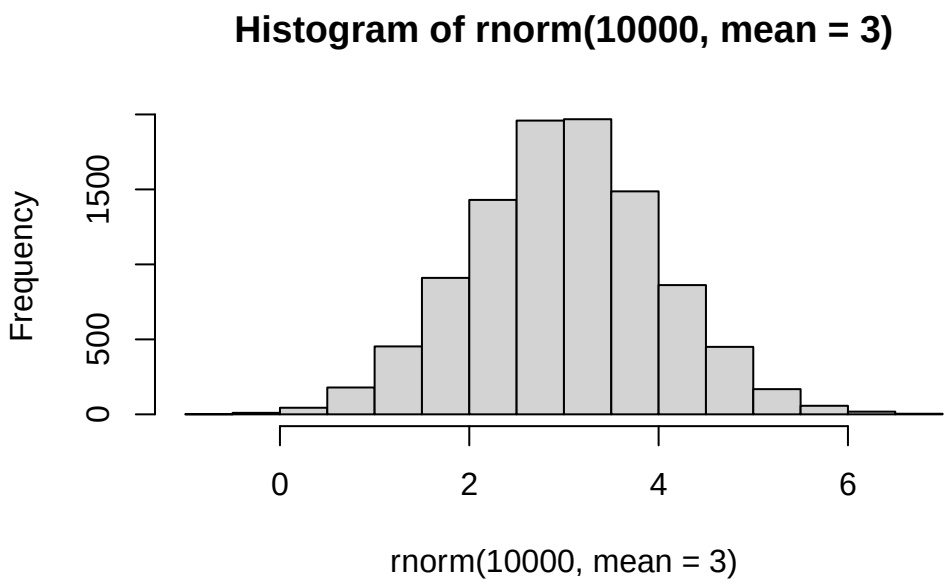
Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionallity reduction**.

K-means clustering

The main function in "base" R for K-means clusterin is called `kmeans()`.

Before we go too deep let's make up some "simple" data that we can cluster and know if we are getting a good answer or not. To do this we can use the `rnorm()` function:

```
hist( rnorm(10000, mean = 3) )
```




```r
rnorm(30, -3)
```

```
 [1] -3.195923 -4.757304 -4.806408 -1.398129 -3.101385 -3.818616 -4.341556
 [8] -2.450225 -3.670913 -3.202308 -3.167574 -3.614648 -3.078421 -6.338496
[15] -3.962889 -2.813964 -2.168893 -2.354745 -1.930789 -1.984976 -3.842752
[22] -3.304430 -2.074110 -5.637955 -2.863432 -2.962255 -2.947124 -1.904103
[29] -3.163845 -1.986980
```

```r
rnorm(30, +3)
```

```
 [1] 3.397275 2.123194 4.649148 3.439960 3.196225 1.943924 2.832314 3.563280
 [9] 4.235730 1.884895 3.070785 3.051182 2.531963 4.679967 3.589710 2.864989
[17] 2.239157 3.589319 5.125102 1.589894 4.324835 3.087057 4.208512 3.118446
[25] 4.088565 3.822317 3.273610 3.237682 4.088230 2.499472
```

```r
x <- c( rnorm(30, -3), rnorm(30, +3) )
x
```

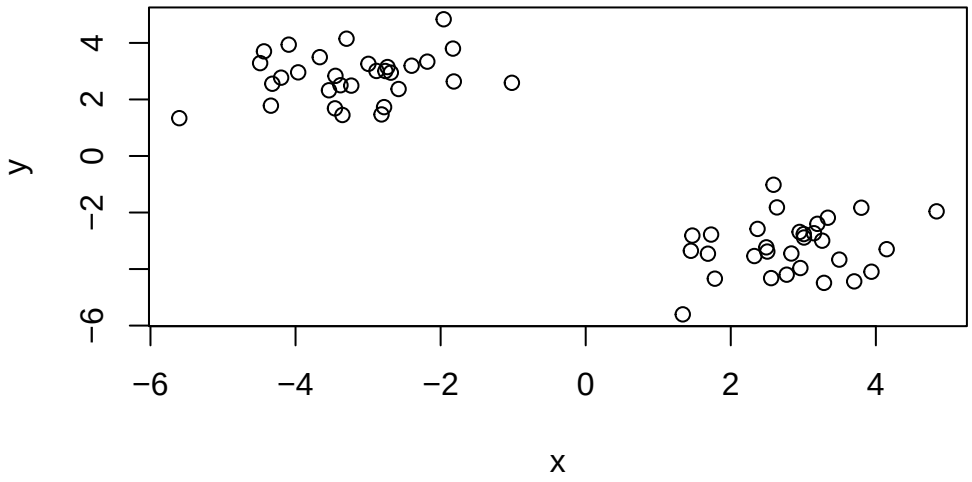
```
 [1] -2.579680 -2.732439 -3.353772 -4.198915 -3.539177 -1.017540 -2.399560
 [8] -1.818276 -2.686596 -3.297111 -4.320167 -4.488112 -3.380447 -4.339869
[15] -4.435298 -2.761822 -3.963618 -4.094059 -2.995773 -3.231680 -3.450137
```

```
[22] -3.666080 -2.183624 -2.814911 -1.958070 -5.602899 -2.779686 -2.884643
[29] -1.829969 -3.456356  1.684867  3.799829  3.009363  1.728760  1.338311
[36]  4.837578  1.470175  3.338065  3.496210  2.834980  2.490445  3.260137
[43]  3.939411  2.959280  3.006397  3.701552  1.780487  2.503591  3.285075
[50]  2.556416  4.149428  2.948871  2.636322  3.193829  2.589231  2.324950
[57]  2.770763  1.449967  3.147011  2.369042
```

```
rev(x)
```

```
 [1]  2.369042  3.147011  1.449967  2.770763  2.324950  2.589231  3.193829
 [8]  2.636322  2.948871  4.149428  2.556416  3.285075  2.503591  1.780487
[15]  3.701552  3.006397  2.959280  3.939411  3.260137  2.490445  2.834980
[22]  3.496210  3.338065  1.470175  4.837578  1.338311  1.728760  3.009363
[29]  3.799829  1.684867 -3.456356 -1.829969 -2.884643 -2.779686 -5.602899
[36] -1.958070 -2.814911 -2.183624 -3.666080 -3.450137 -3.231680 -2.995773
[43] -4.094059 -3.963618 -2.761822 -4.435298 -4.339869 -3.380447 -4.488112
[50] -4.320167 -3.297111 -2.686596 -1.818276 -2.399560 -1.017540 -3.539177
[57] -4.198915 -3.353772 -2.732439 -2.579680
```

```
z <- cbind(x=x, y=rev(x))
plot(z)
```



```r
p <- 1:5
rbind(p, p)
```

```
  [,1] [,2] [,3] [,4] [,5]
p    1    2    3    4    5
p    1    2    3    4    5
```

Now we can run `kmeans()` on this input `z` and see what the results look like.

```r
km <- kmeans(z, centers = 2)
km
```

```
K-means clustering with 2 clusters of sizes 30, 30

Cluster means:
          x         y
1 -3.208676  2.820011
2  2.820011 -3.208676

Clustering vector:
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2

Within cluster sum of squares by cluster:
[1] 48.29678 48.29678
 (between_SS / total_SS =  91.9 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```r
attributes(km)
```

```
$names
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"

$class
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What "component of your result object details cluster assigment/membership?

```
km$cluster
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

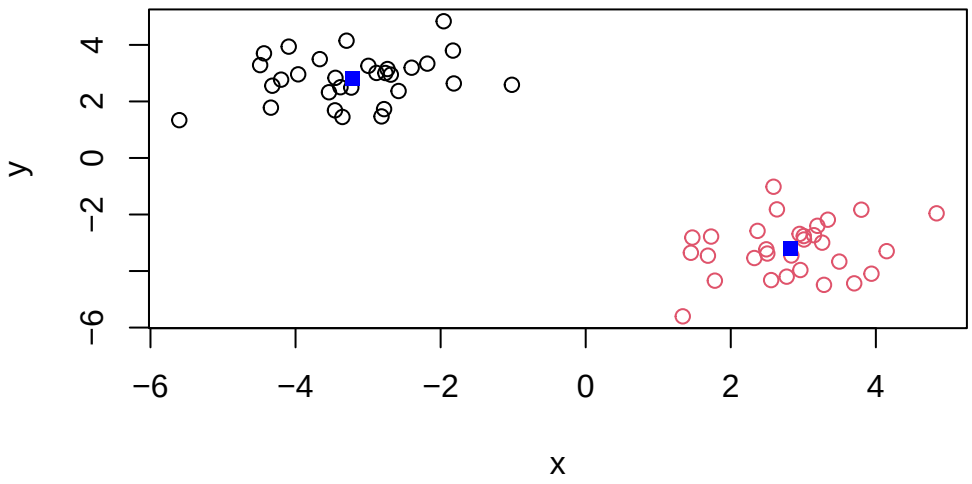
Q. What "component of your result object details cluster center?

```
km$centers
```

```
          x         y
1 -3.208676  2.820011
2  2.820011 -3.208676
```

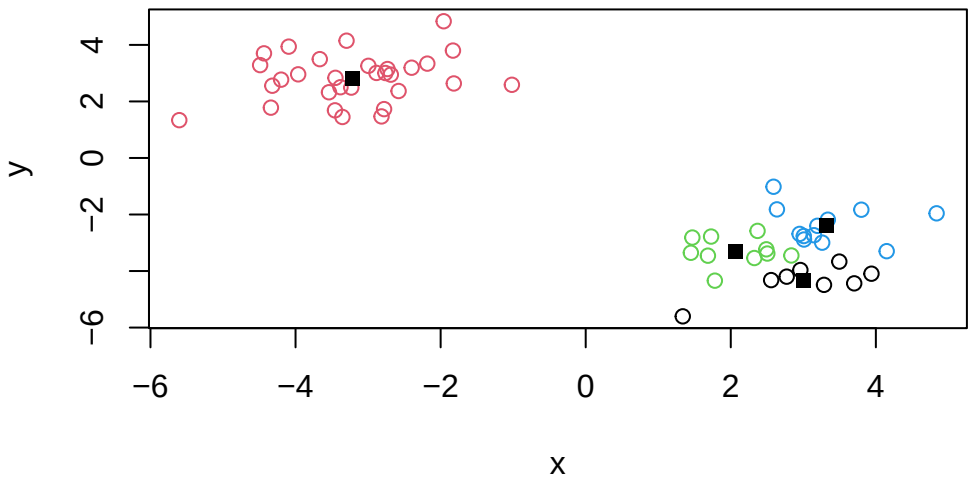
Q. Plot `z` colored by the kmeans cluster assignment and add cluster centers as blue points.

```
plot(z, col = km$cluster)
points(km$centers, col = "blue", pch = 15)
```



Q. Run a K-means clustering and plot the results asking for 4 clusters?

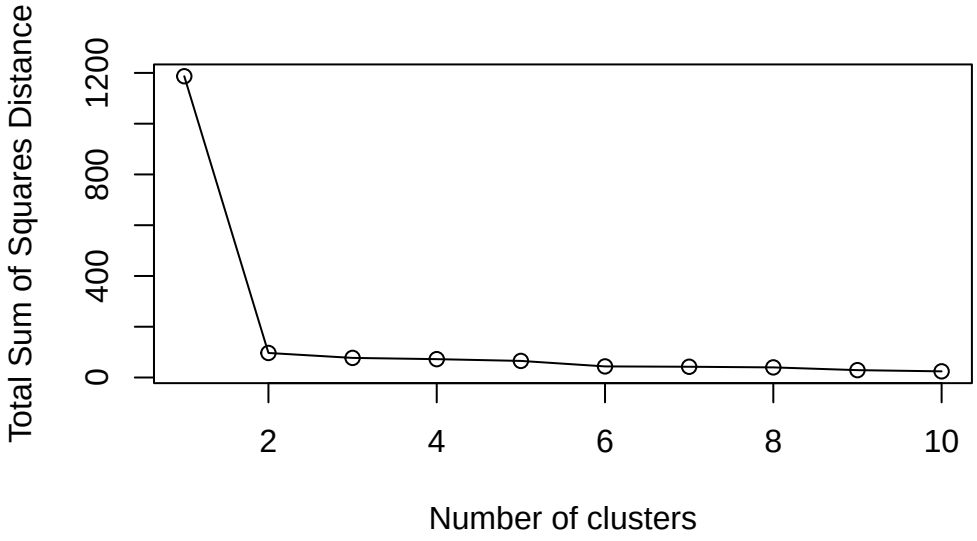
```
km4 <- kmeans(z, centers = 4)
plot(z, col = km4$cluster)
points(km4$centers, col = "black", pch = 15)
```



N.B. You need to tell K-means the number of clusters (i.e. set `centers=2`)!!

One approach is to try different values for `centers` and then pick the best…

```
ans <- NULL
for(i in 1:10) {
  km <- kmeans(z, centers = i)
  ans <- c(ans, km$tot.withinss)
}

plot(ans, type = "o", xlab = "Number of clusters", ylab = "Total Sum of Squares Distance")
```



Hierarchical Clustering

The main function in "base" R for Hierarchical Clustering is called `hclust()`

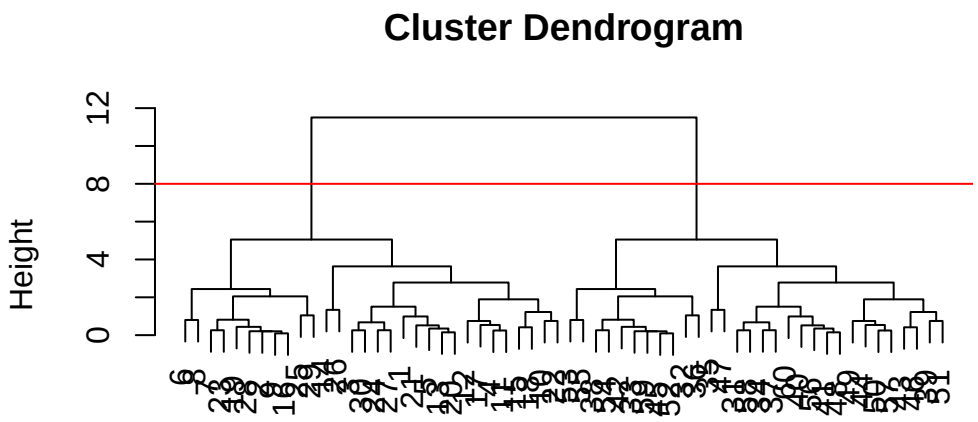
This function does not take your "raw" data for clustering. You must first build a "distance matrix" from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

```
Call:
hclust(d = d)

Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)
abline(h = 8, col = "red")
```



Once we hav eour `hclust()` object (our "tree" of "cluster dendrogram") we can *"cut"* the tree to reveal the clustering pattern.

```
cutree(hc, h = 8)
```

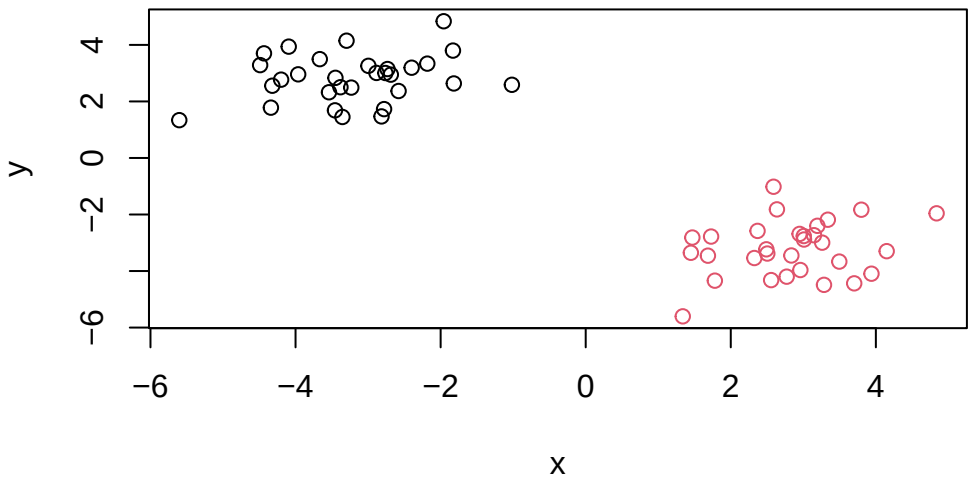
```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

```
cutree(hc, k = 4)
```

```
 [1] 1 2 1 1 1 2 2 2 2 1 1 1 1 1 1 2 1 1 2 1 1 1 2 1 2 1 1 2 2 1 3 4 4 3 3 4 3 4
[39] 3 3 3 4 3 3 4 3 3 3 3 3 3 4 4 4 4 3 3 3 4 3
```

Q. Make a plot of `z` with your hclust results (i.e. colored by cluster membership)

```
grps <- cutree(hc, k = 2)
plot(z, col = grps)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets.

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data Import

The data is made available in CSV format so we can use the `read.csv()` function for import to R:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```

```
                    X England Wales Scotland N.Ireland
1              Cheese     105   103      103        66
2        Carcass_meat     245   227      242       267
3          Other_meat     685   803      750       586
4                Fish     147   160      122        93
5       Fats_and_oils     193   235      184       209
6             Sugars     156   175      147       139
7      Fresh_potatoes     720   874      566      1033
8          Fresh_Veg     253   265      171       143
9          Other_Veg     488   570      418       355
10 Processed_potatoes     198   203      220       187
11     Processed_Veg     360   365      337       334
12       Fresh_fruit    1102  1137      957       674
13           Cereals    1472  1582     1462      1494
14         Beverages      57    73       53        47
15       Soft_drinks    1374  1256     1572      1506
16   Alcoholic_drinks     375   475      458       135
17      Confectionery      54    64       62        41
```

Tidy the data

Fix anything that went wrong with data import.

> Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17  5
```

```
head(x)
```

```
              X England Wales Scotland N.Ireland
1        Cheese     105   103      103        66
2  Carcass_meat     245   227      242       267
3    Other_meat     685   803      750       586
4          Fish     147   160      122        93
5 Fats_and_oils     193   235      184       209
6        Sugars     156   175      147       139
```

```
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

```
              England Wales Scotland N.Ireland
Cheese            105   103      103        66
Carcass_meat      245   227      242       267
Other_meat        685   803      750       586
Fish              147   160      122        93
Fats_and_oils     193   235      184       209
Sugars            156   175      147       139
```

```
dim(x)
```

```
[1] 17  4
```

```
x <- read.csv(url, row.names=1)
head(x)
```

```
              England Wales Scotland N.Ireland
Cheese            105   103      103        66
Carcass_meat      245   227      242       267
Other_meat        685   803      750       586
Fish              147   160      122        93
Fats_and_oils     193   235      184       209
Sugars            156   175      147       139
```

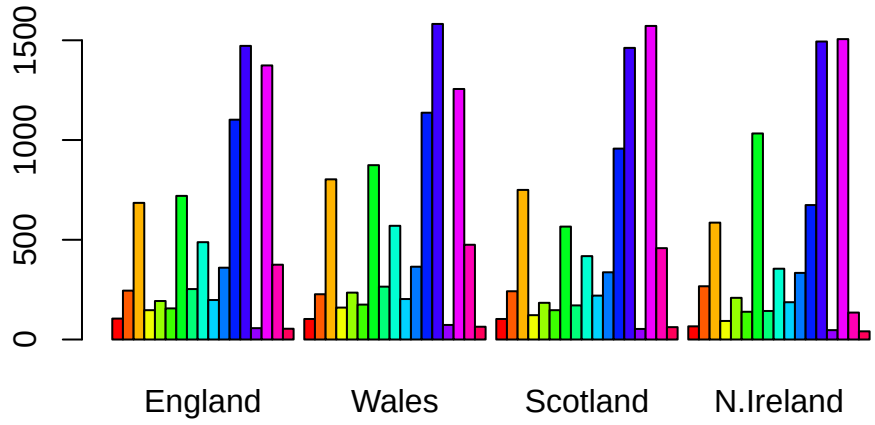
> Q2. Which approach to solving the 'row-names problem' mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

The approach of setting the row.names() argument is better. The other approach using x <- x[,-1] causes the data frame to be cut down every time the command is run. When dealing with especially large dataframes, it will be much more efficient to set the row.names() argument.

Exploratory analysis

Make some plots to help make sense of obvious trends...

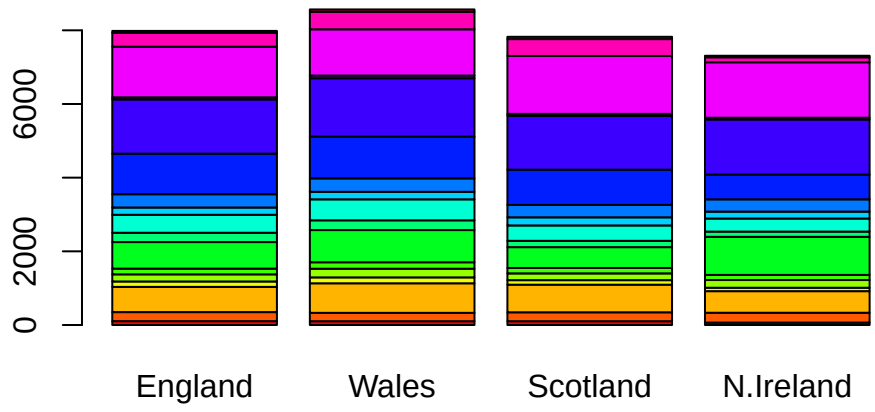
```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above barplot() function results in the following plot?

Changing the "beside" argument results in the following plot.

```
barplot(as.matrix(x), beside=FALSE, col=rainbow(nrow(x)))
```




```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
          tibble::rownames_to_column("Food") |>
          pivot_longer(cols = -Food,
                       names_to = "Country",
                       values_to = "Consumption")

dim(x_long)
```

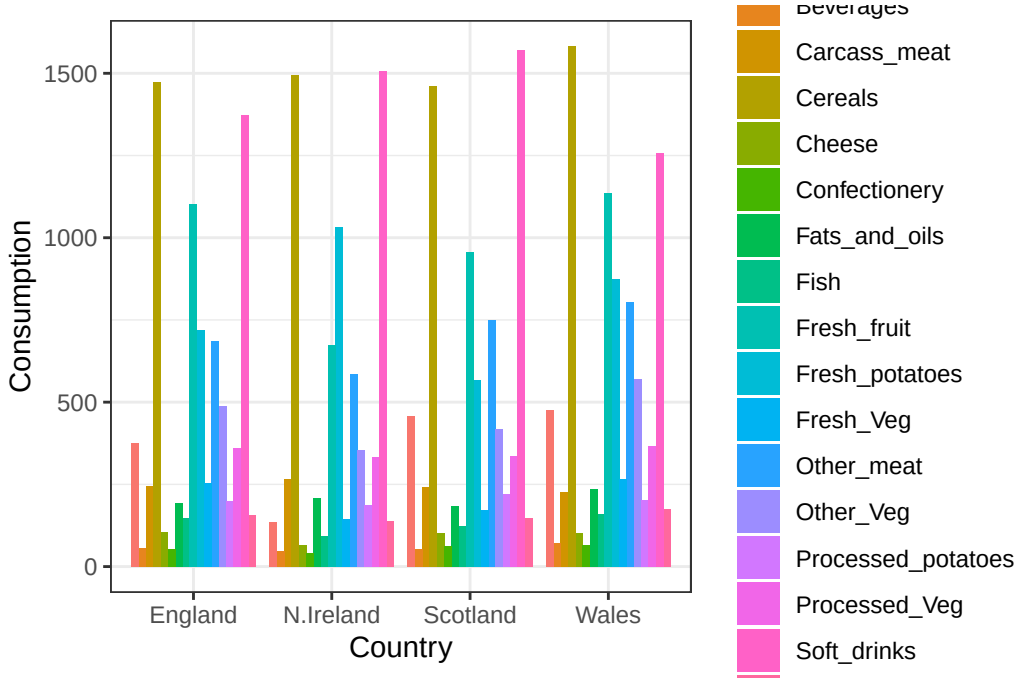
```
[1] 68  3
```

```
head(x_long)
```

```
# A tibble: 6 x 3
  Food            Country   Consumption
  <chr>           <chr>           <int>
1 "Cheese"        England           105
2 "Cheese"        Wales             103
3 "Cheese"        Scotland          103
```

```
4 "Cheese"        N.Ireland          66
5 "Carcass_meat " England           245
6 "Carcass_meat " Wales             227
```

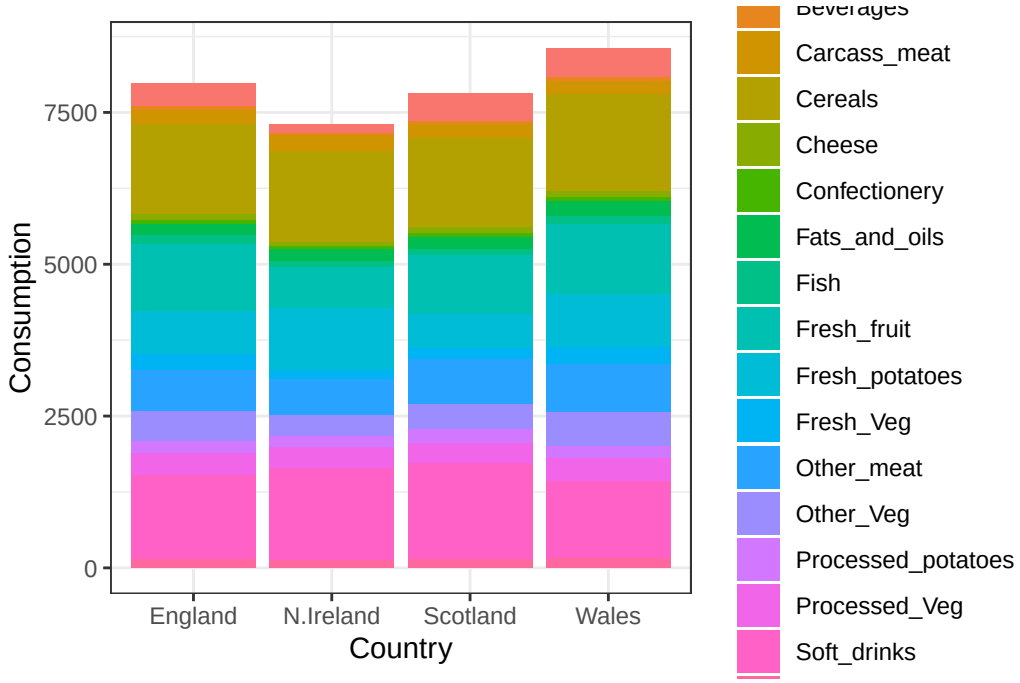
```
library(ggplot2)
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



Q4: Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

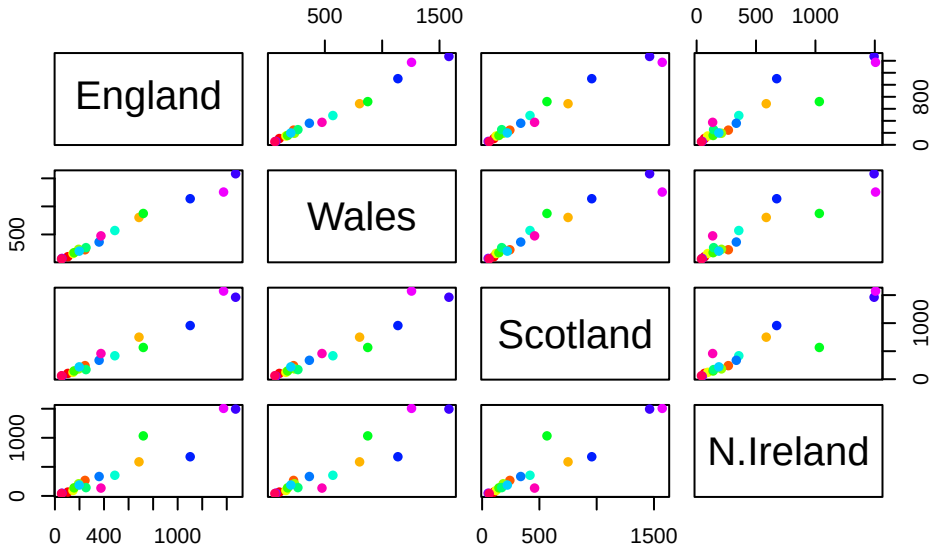
Changing the geom_col() function's from position = "dodge" to position = "stack" results in a stacked barplot figure.

```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```



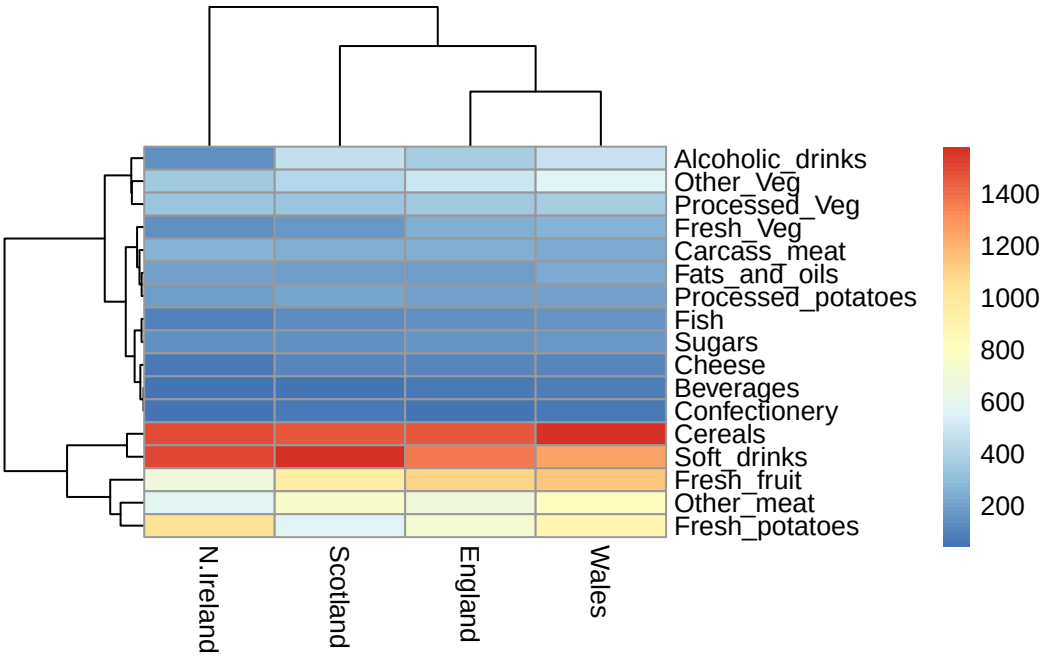
Q5: We can use the pairs() function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



The orientation of country name determines the x and y values. For example, the plots in the top row have England as the y-axis and the x-axis is Wales, Scotland, and N. Ireland, respectively. If a given point lies on the diagonal, it indicates a perfect correlation between the two variables.

```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

Based on the figures, England and Wales seem to cluster together, suggesting that they have similar food consumption patterns, particularly in having less consumption of specific meats. The heatmap shows that the main differences between N. Ireland and the other countries are lower alcoholic consumption, lower fresh fruit, and more "other meat".

Key-point: Even relatively small datasets can prove challenging to interpret.

PCA to the rescue

The main function in "base" R for PCA is called `prcomp()`. This function wants the "observations" to be rows and the "variables" to be columns.

So here we need to take the transpose of our `x` input object.

```
pca <- prcomp(t(x))
summary(pca)
```

```
Importance of components:
                            PC1      PC2      PC3       PC4
Standard deviation     324.1502 212.7478 73.87622 3.176e-14
Proportion of Variance   0.6744   0.2905  0.03503 0.000e+00
Cumulative Proportion    0.6744   0.9650  1.00000 1.000e+00
```

The returned `pca` object has commponents that we can use to make our main result figures:

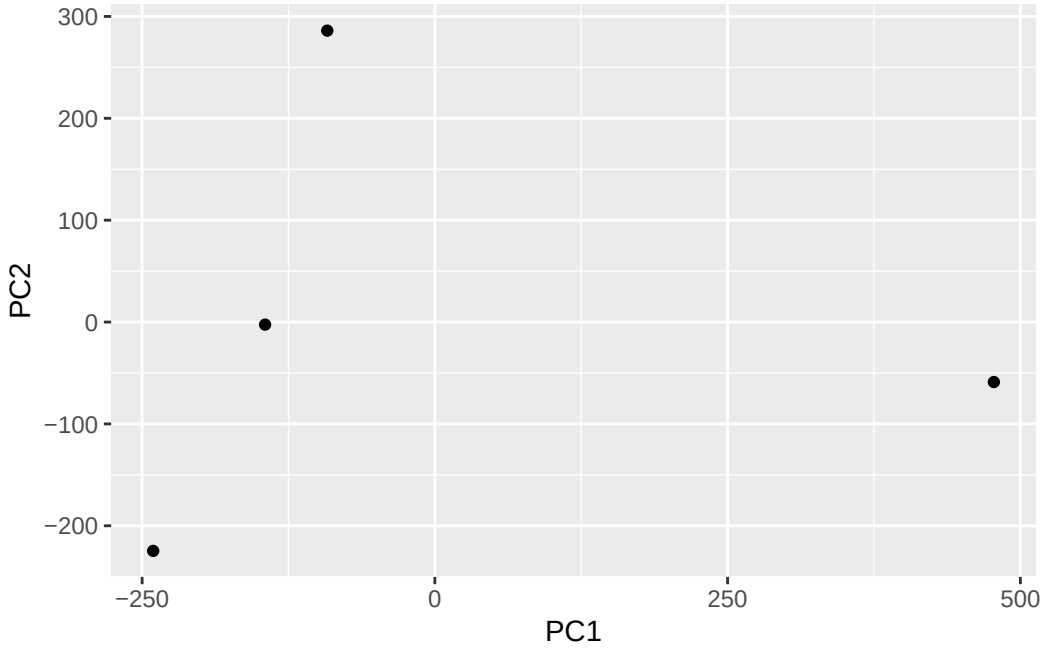
```
attributes(pca)
```

```
$names
[1] "sdev"     "rotation" "center"   "scale"    "x"

$class
[1] "prcomp"
```

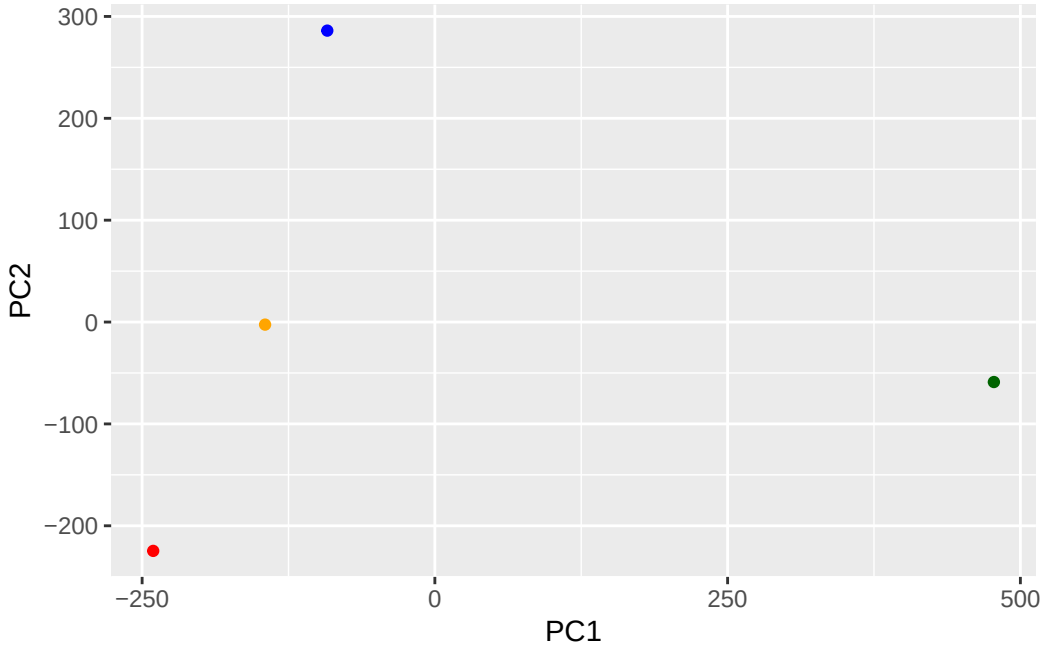
The main result figure from this analysis is called a **"PC score plot"** (a.k.a. an "ordination plot", "PC plot" or simply "PC1 vs PC2 plot").

This plot shows how samples (n this case countries) relate to each other.

```
library(ggplot2)

ggplot(pca$x) +
  aes(PC1, PC2) +
  geom_point()
```



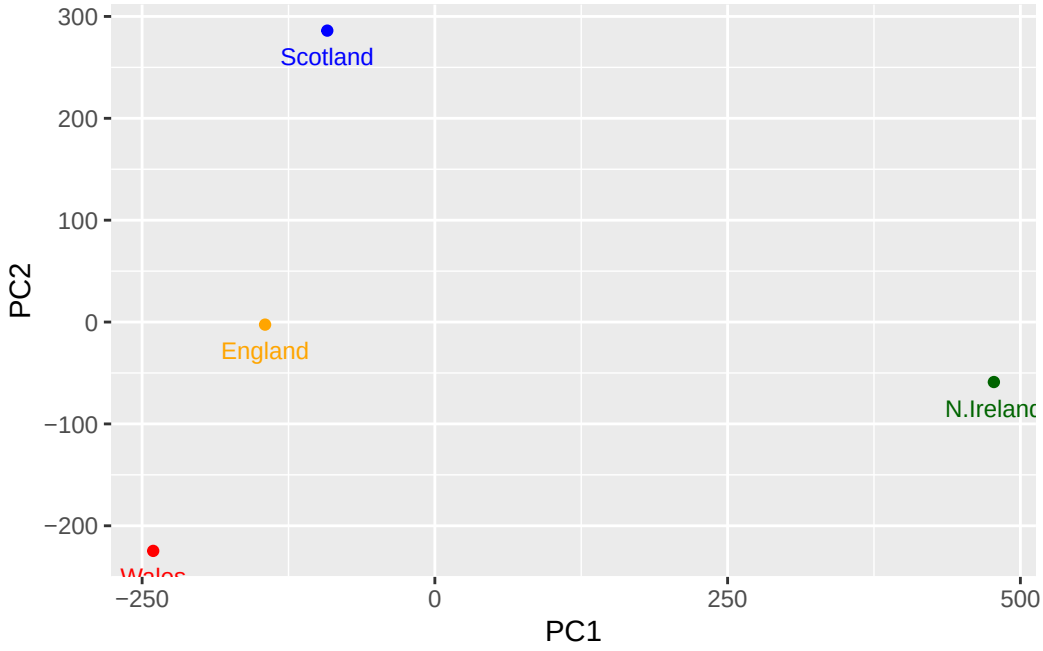

```
mycols <- c("orange", "red", "blue", "darkgreen")

ggplot(pca$x) +
  aes(PC1, PC2) +
  geom_point(col = mycols)
```



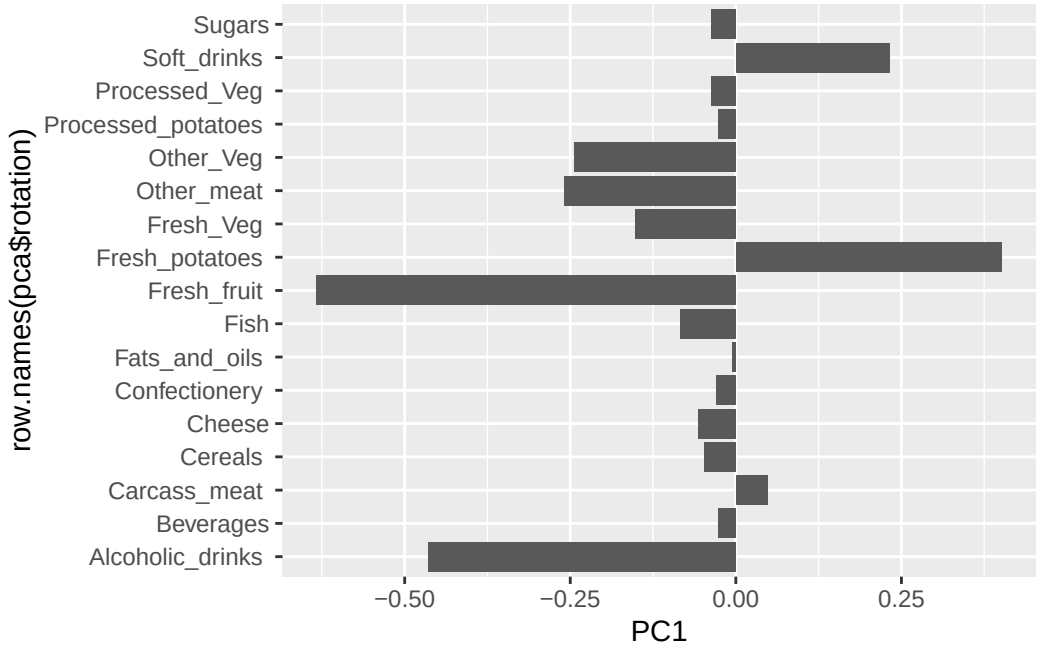

```
pca$x
```

```
                PC1         PC2        PC3           PC4
England   -144.99315   -2.532999 105.768945 -4.894696e-14
Wales     -240.52915 -224.646925 -56.475555  5.700024e-13
Scotland   -91.86934  286.081786 -44.415495 -7.460785e-13
N.Ireland  477.39164  -58.901862  -4.877895  2.321303e-13
```

```
ggplot(pca$x) +
  aes(PC1, PC2, label=row.names(pca$x)) +
  geom_point(col = mycols) +
  geom_text(size = 3, vjust = 2, col = mycols)
```




```
ggplot(pca$rotation) +
  aes(PC1, row.names(pca$rotation)) +
  geom_col()
```



Note: In PCA, score plots and loading plots focus on providing more in-depth visualizations. Score plots focus on the observations, or the samples, while loading plots focus on the variables.